

Data Structures & Algorithms Using C

An industry-oriented syllabus designed to build strong programming fundamentals, sharpen problem-solving skills, and prepare students for software development roles, coding interviews, and product-based company placements.

UNDERGRADUATE PROGRAM

C PROGRAMMING

10 HOURS TOTAL



What This Course Covers

This syllabus bridges the gap between academic theory and real-world software engineering. Every topic is chosen to give you the skills that interviewers and engineering teams actually look for.

Coding Interviews

Master the patterns and problem-solving techniques used by top product companies like Google, Amazon, and Microsoft.

Competitive Programming

Build speed and accuracy in solving algorithmic challenges under time constraints on platforms like Codeforces and LeetCode.

Software Development

Understand how data structures underpin real systems — from databases and operating systems to web servers and compilers.

C Language Mastery

Learn C as the foundation of systems programming, giving you direct control over memory and performance.



MODULE 1

Introduction to Programming & Problem Solving

Duration: 4 Hours — This module lays the conceptual groundwork. Before writing a single line of C code, you'll learn how to think like a programmer: breaking problems down, designing solutions, and evaluating their efficiency.

Module 1 – Topics at a Glance

Core Concepts

- What are Algorithms and Data Structures?
- Problem-Solving Methodology
- Flowcharts and Pseudocode
- Time and Space Complexity Basics

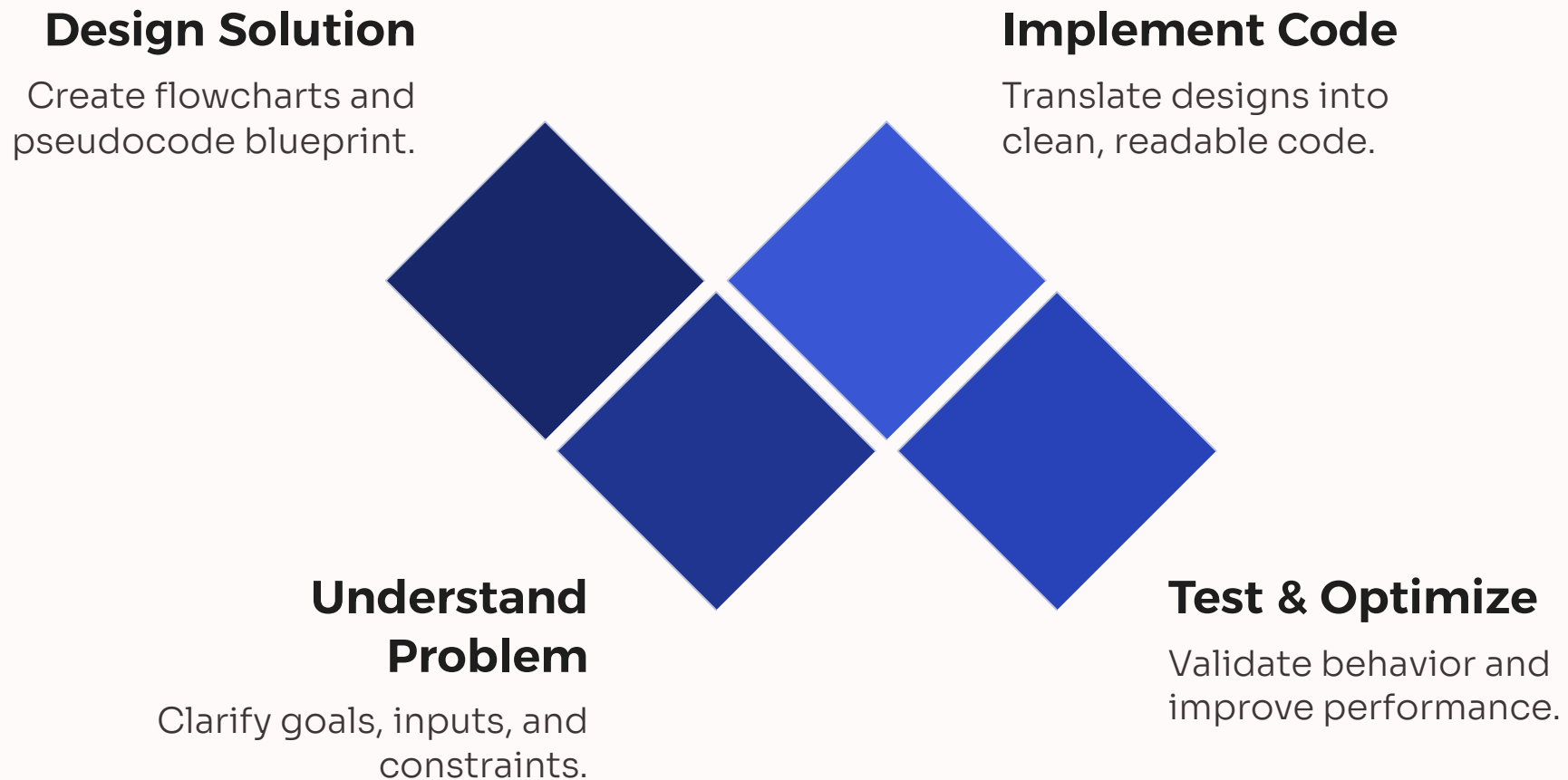
C Programming Setup

- Introduction to the C Programming Environment
- Setting up a compiler (GCC, IDE)
- Writing your first C program
- Input/Output Operations (printf, scanf)

📌 Understanding complexity early helps you write efficient code from day one — a skill that separates good engineers from great ones.

Thinking Before Coding: Problem-Solving Methodology

Great engineers don't jump straight to code. They follow a disciplined process to understand, plan, and solve problems systematically.



Flowcharts and pseudocode are your blueprints — they help you reason about logic before committing to syntax. This approach is universally used in technical interviews and real engineering teams.

Time & Space Complexity – The Language of Efficiency

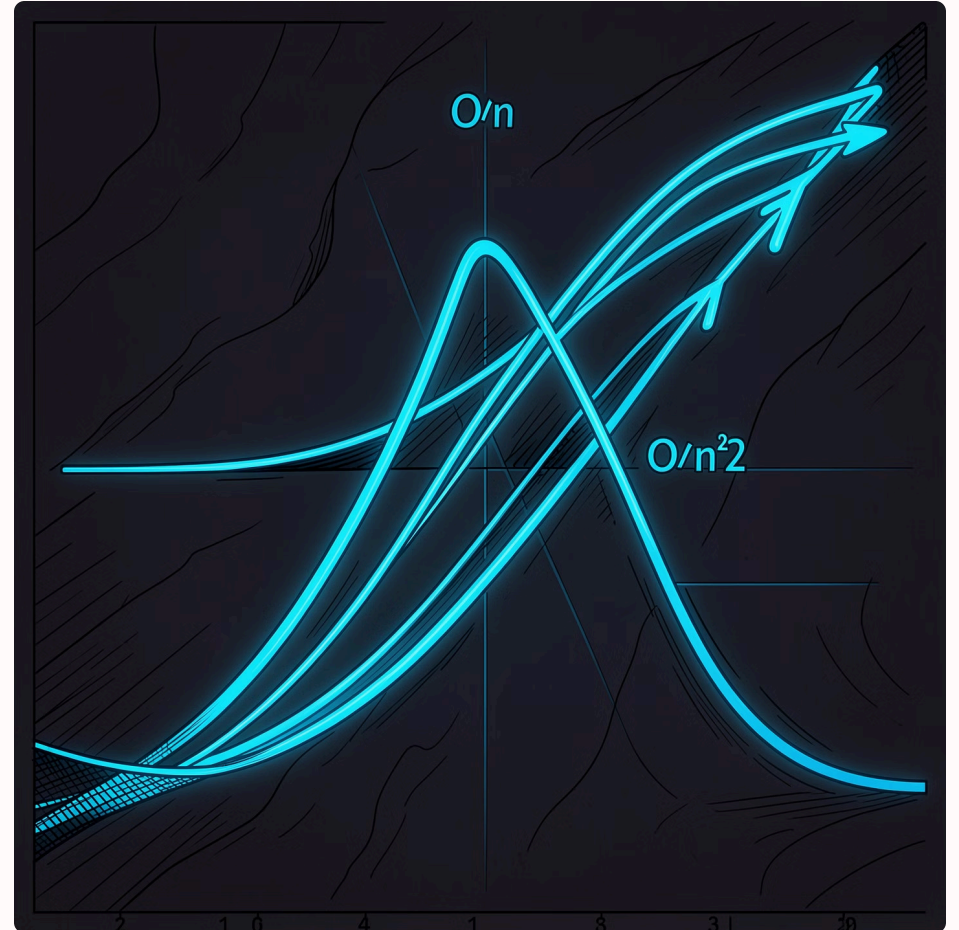
Complexity analysis tells you how your code scales as input grows. Interviewers use it to assess whether you understand the cost of your solutions.

Time Complexity

Measures how runtime grows with input size. Expressed using Big-O notation: $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$.

Space Complexity

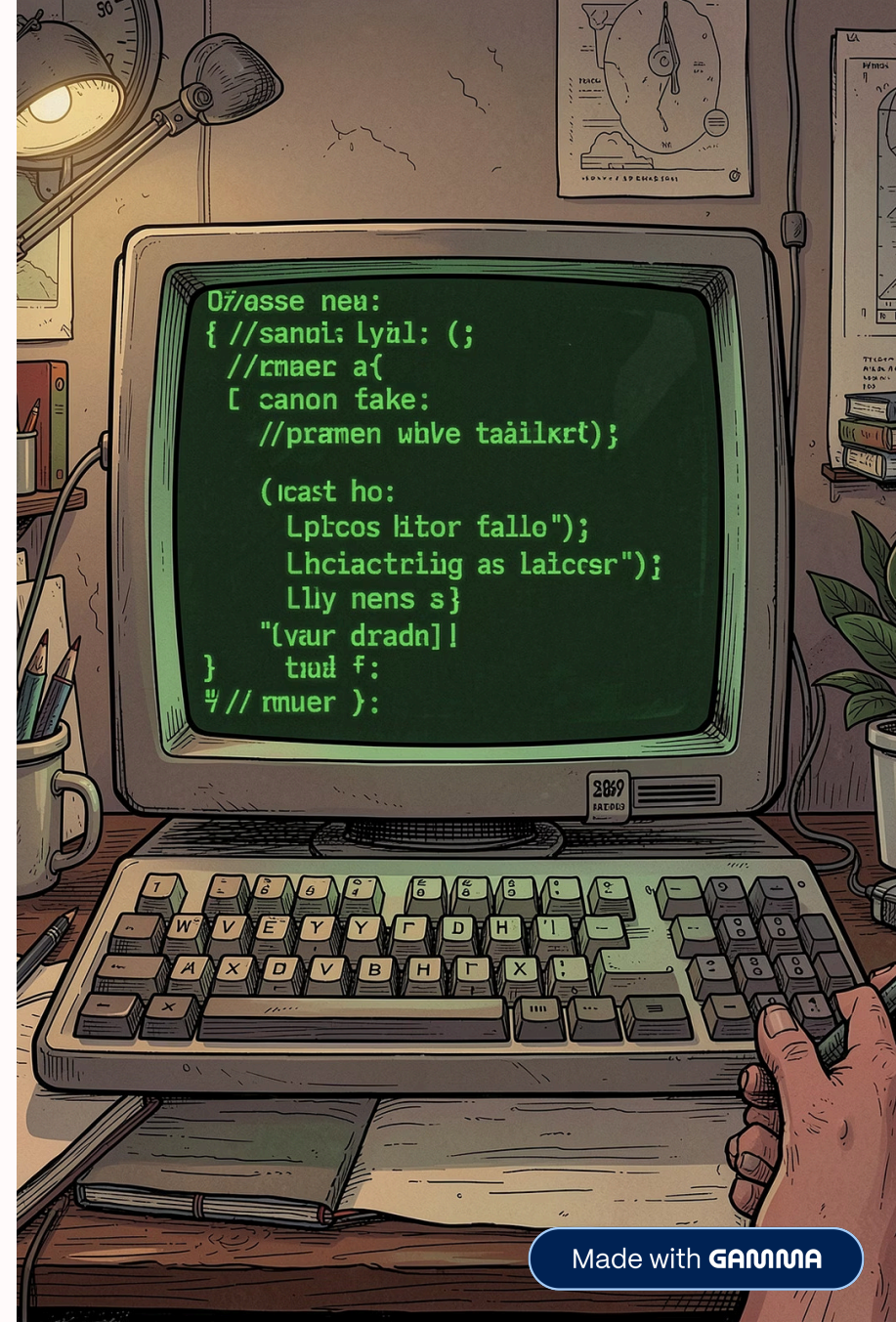
Measures how much memory your algorithm consumes. Critical in embedded systems and large-scale applications.



MODULE 2

C Programming Fundamentals

Duration: 6 Hours — C is the mother of modern programming languages. Mastering it gives you a deep understanding of how computers actually work — memory, pointers, and all. This module is the technical core of the course.



Module 2 – Building Blocks of C



Variables & Data Types

int, float, char, double — and how operators manipulate them. The foundation of every C program.



Conditionals & Loops

if/else, switch, for, while, and do-while. Control the flow of your program with precision.



Functions & Recursion

Modular code design, parameter passing, return values, and the elegance (and danger) of recursive solutions.



Arrays & Strings

Contiguous memory blocks, indexing, character arrays, and string manipulation — essential for interview problems.

Pointers, Memory & Structures – The C Superpowers

Pointers & Memory Management

Pointers are C's most powerful — and most feared — feature. Understanding them unlocks dynamic memory allocation, efficient array handling, and the ability to work with data structures like linked lists and trees.

- Pointer arithmetic and dereferencing
- malloc, calloc, free — manual memory control
- Common pitfalls: dangling pointers, memory leaks

Structures & Unions

Group related data into meaningful units. Structures are the blueprint for custom data types — the precursor to classes in object-oriented languages.

- Defining and using struct types
- Nested structures and arrays of structs
- Unions and when to use them
- typedef for cleaner code

Course Summary & What Comes Next

These two modules form the essential foundation. From here, the full DSA journey continues into stacks, queues, linked lists, trees, graphs, sorting, and searching — all built on the C fundamentals you've now established.

10

Total Hours

Across 2 focused modules

14+

Core Topics

From complexity to pointers

1

Language

C — the foundation of systems programming

4

Career Paths

Interviews, CP, dev, and placements

📌 **Next Steps:** Practice daily on platforms like LeetCode and HackerRank. Revisit pointer concepts until they feel intuitive. Build small C projects to cement your understanding before moving to advanced data structures.